

Python Q&A Assistant

Grounded answers to Python questions for data-science learners — with citations, served over a REST API.

- Retrieval-augmented generation over a curated Stack Overflow corpus
- Every answer cites real sources; refuses when it has no basis
- FastAPI service, local embeddings, zero external infra beyond the LLM

3-day take-home · LangChain · FAISS · Groq Llama-3.3-70b

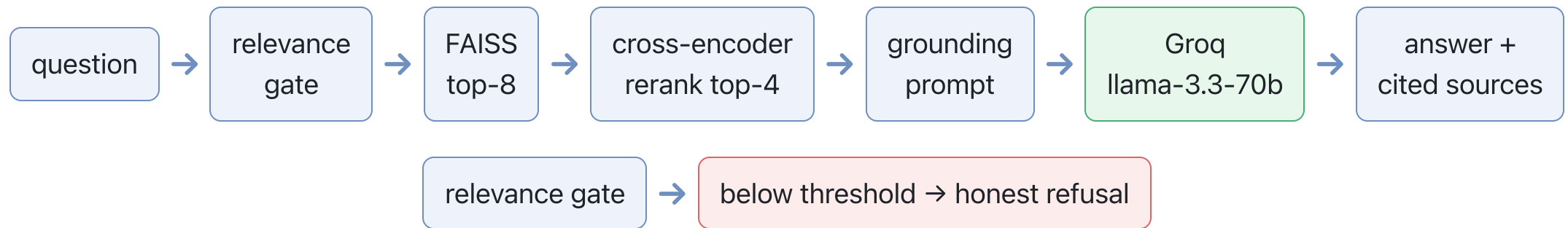
Problem & approach

Problem. Learners ask the same Python questions that are already answered well on Stack Overflow. Raw LLMs answer confidently but hallucinate APIs — bad when someone is still building a mental model.

Approach. RAG over a curated Stack Overflow Q&A corpus, so every answer is:

- **Grounded** in real, highly-voted sources
- **Cited** — links back to the questions it used
- **Honest** — it refuses when retrieval finds no basis, instead of bluffing

Architecture



- Embed the question, check it matches the index (refuse if not)
- Retrieve **broad** with the bi-encoder, rerank **precise** with the cross-encoder
- **Local embeddings + local vector store = zero external infra beyond the LLM**

Data pipeline

- **Source:** Kaggle Stack Overflow Python dump (~1.7 GB, millions of rows)
- Pair each question with its **highest-scored answer** (accepted-flag proxy)
- **Quality filter** (score ≥ 5), cap to ~50k pairs
- **Preserve code blocks as fenced markdown** through cleaning + chunking

For a Python assistant, flattening code into prose throws away most of the value — **87% of docs contain code**, so this got real attention in `ingest.py`.

Retrieval

- `bge-small-en-v1.5` embeddings, normalized; query-side instruction prefix
- Retrieve **broad (top-8)**, rerank **precise (top-4)** with a cross-encoder

Why broad-then-precise? The cross-encoder reads the full `(query, passage)` pair, so it orders results far better than cosine similarity alone. The bi-encoder gets recall cheaply; the cross-encoder buys precision where it matters.

Generation — two-layer honesty

Grounding prompt: answer *only* from context, prefer runnable code, cite sources by number, say "I don't know" when the context doesn't cover it.

Layer 1 — relevance gate

Score the question against the index; refuse *below threshold* before the LLM is ever called.

Layer 2 — the prompt

Even if a question slips past the gate, the prompt makes the LLM decline when retrieval was weak.

Refusal is enforced, not "asked for nicely." That's what makes it honest.

API

- **FastAPI**, async `POST /ask` and a real `GET /health` (reports index status + vector count for platform health checks)
- Index + models loaded **once on startup** (lifespan), never per request
- Blocking work offloaded to a thread, so the event loop stays free
- **TTL cache** on the normalised question; **422** on empty input via pydantic

```
curl -X POST localhost:8000/ask -H 'Content-Type: application/json' \
  -d '{"question":"how do I reverse a list in python"}'
```

Testing & eval

- **pytest**: API happy path, validation (422), cache hit, refusal — LLM mocked, plus one live integration test. *10 passed, 1 skipped*.
- **10 documented eval queries** with real transcripts (`eval.ipynb`)

Honest failure case: pandas questions get **refused** on the small demo index — a *coverage gap, not a retrieval bug*. The cross-encoder orders candidates well; there simply aren't pandas docs in a 150-question slice. Fixed by indexing the full corpus (`INDEX_MAX_DOCS=0`).

Scaling to 100+ concurrent users

Measured locally: cold answer ~12 s (model warmup) · warm ~1–2 s · **cache hit ~1 ms**. Groq latency dominates the warm path; embedding + rerank are a few hundred ms on CPU.

Lever	Effect
Caching (in-mem → Redis, shared across workers)	50% hit rate halves LLM calls; those drop ~1–2 s → ~1 ms
Async + N uvicorn workers (gunicorn, $N \approx \text{cores}$)	one box from ~1 RPS to tens
Batch query embedding under load	one encode call for concurrent queries
Externalise state (Qdrant/Pinecone + Postgres)	autoscale stateless API replicas behind an LB
Cost (cache hit-rate, cheaper LLM for easy Qs)	fewer tokens = lower bill

Back-of-envelope: Redis @ ~50% hit + 4 workers / 4-core box comfortably serves 100+ users on mostly repeated traffic; bursty unique traffic scales by adding replicas.

Tradeoffs & next steps

- **Chose zero-infra simplicity** (FAISS, local embeddings) over scale-out readiness — fine for a demo, swappable for prod (retriever-level change, not a rewrite)
- Demo ships a **small index** for speed; the full corpus is one env var away
- The relevance gate is tuned to the small index — needs re-tuning for the full one

Next: full ~50k index + re-tuned gate · retrieval metrics (hit@k / MRR) · query rewrite for one-word questions ·

Redis + workers for real concurrency